Семинар 6. Модели согласованности, сериализуемость и транзакционность

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- (Напоминание) Второе домашнее задание – жесткий дедлайн в 23:59 02.04.2026
- Исправления по первому домашнему заданию
  - ≥ 0.75 – исправления не принимаются ("4" за задание есть)
  - Просьба: не исправляйте! Делайте следующее задание :)

Содержание

1. Модели согласованности
2. Сериализуемость транзакций
3. Транзакционность

Понятие согласованности

Модель согласованности

Модель системы, которая определяет, какие значения при чтении могут быть возвращены после записей.

- На практике выражается как:
  - Контракты и гарантии API к хранилищу данных
  - Транзакционные уровни изоляции в СУБД
  - Гарантии доставки сообщений в системах обмена сообщениями
- Возникает и в других разделах computer science:
  - Модели памяти в языках программирования (Java Memory Model, C++11)
- Выделяют более *сильные* и более *слабые* модели согласованности
  - Чем слабее модель, тем проще масштабироваться и выше доступность
  - Чем сильнее модель, тем меньше аномалий, но выше задержка

Consistency vs. Integrity

Consistency (согласованность)

- Какие допустимые состояния системы видны нам после изменения
- Должны ли мы видеть обновление? С какой допустимой задержкой? В каком порядке?
- Пример: read-your-writes (session consistency)

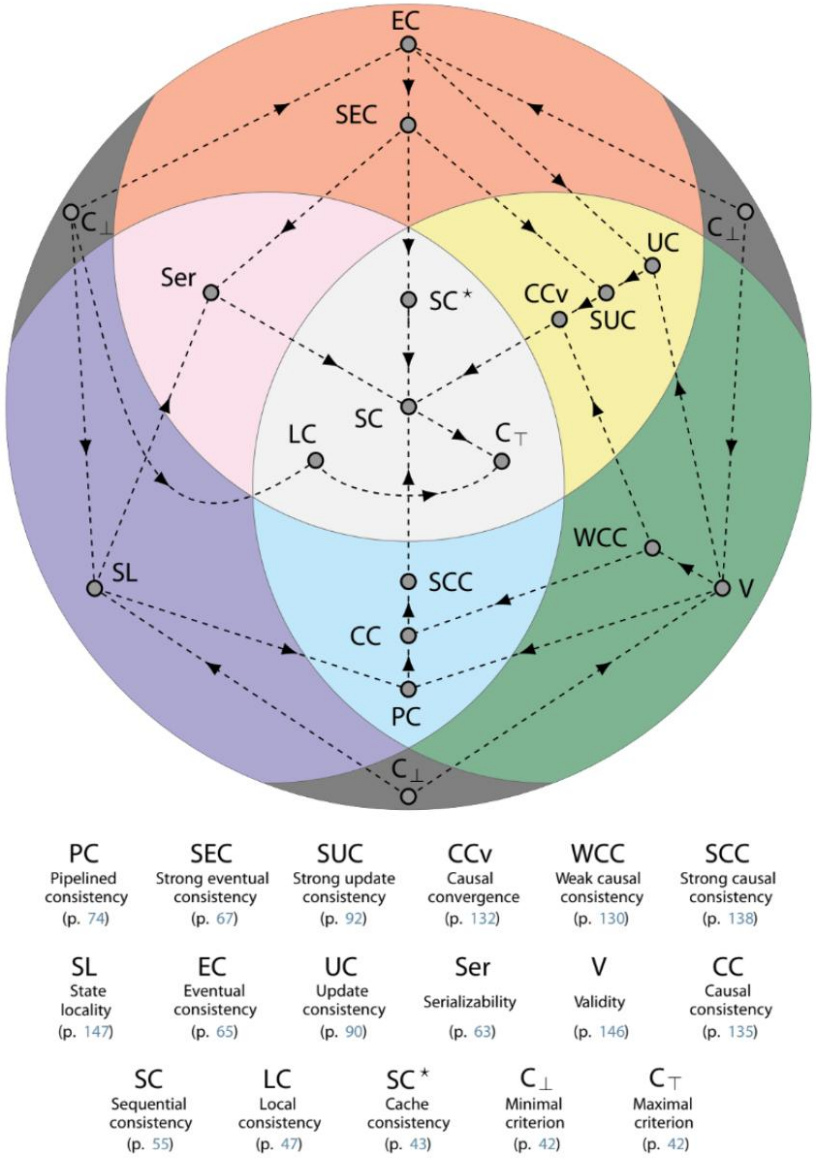
Integrity (целостность)

- Какие состояния запрещены инвариантами
- Пример: `balance >= 0`, уникальность email

Важно

По-русски «согласованность» и «целостность» иногда путают, но это разные вещи.

Некоторые важные модели по M. Perrin



1. **Strong Consistency**
   Все операции читают последнее записанное значение
2. **Sequential Consistency**
   Существует общий линейный порядок операций
3. **Causal Consistency**
   Happens-before порядок сохраняется
4. **Eventual Consistency**
   Спустя какое-то время реплики сойдутся
5. **Strong Eventual Consistency**
   Реплика = множество полученных ей сообщений

Спектр моделей согласованности

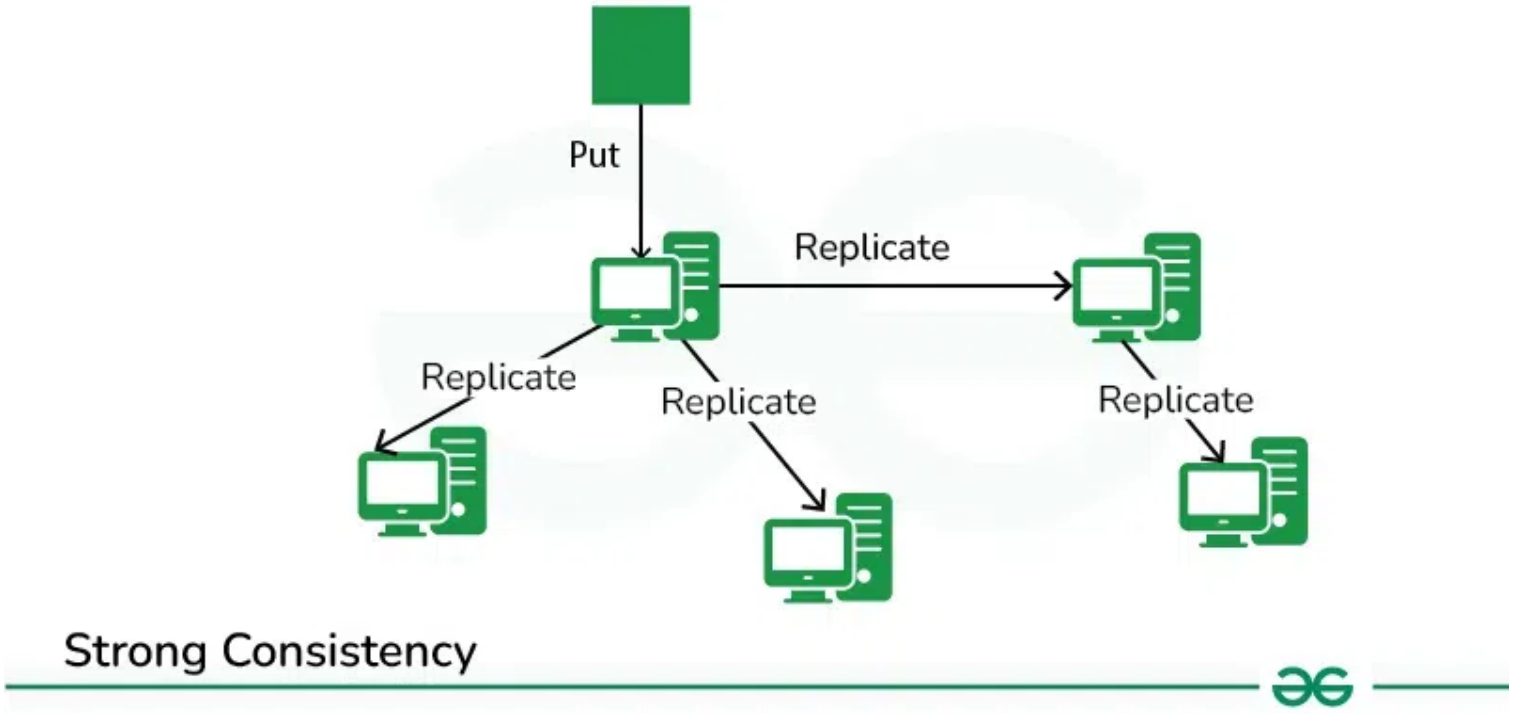
Сильные модели

- Strong Consistency
- Sequential Consistency

Слабые модели

- Eventual Consistency
- Strong Eventual Consistency
- Causal Consistency
- Session Consistency

Strong Consistency



- «Одна копия»: Все узлы видят операции в одном глобальном порядке, согласованном с wall-clock временем
- + Лучше некуда
- – Высокая latency из-за коммуникации (консенсус)
- Для денег, персональных данных, критичных бизнес-инвариантов

Sequential Consistency

- Схема репликации такая же, как и со strong consistency, но:
  - Операция считается принятой **до** тех пор, пока она будет применена на всех репликах
  - Но все еще существует единый глобальный порядок всех операций
  - Порядок не обязан совпадать с wall-clock временем

Следствие

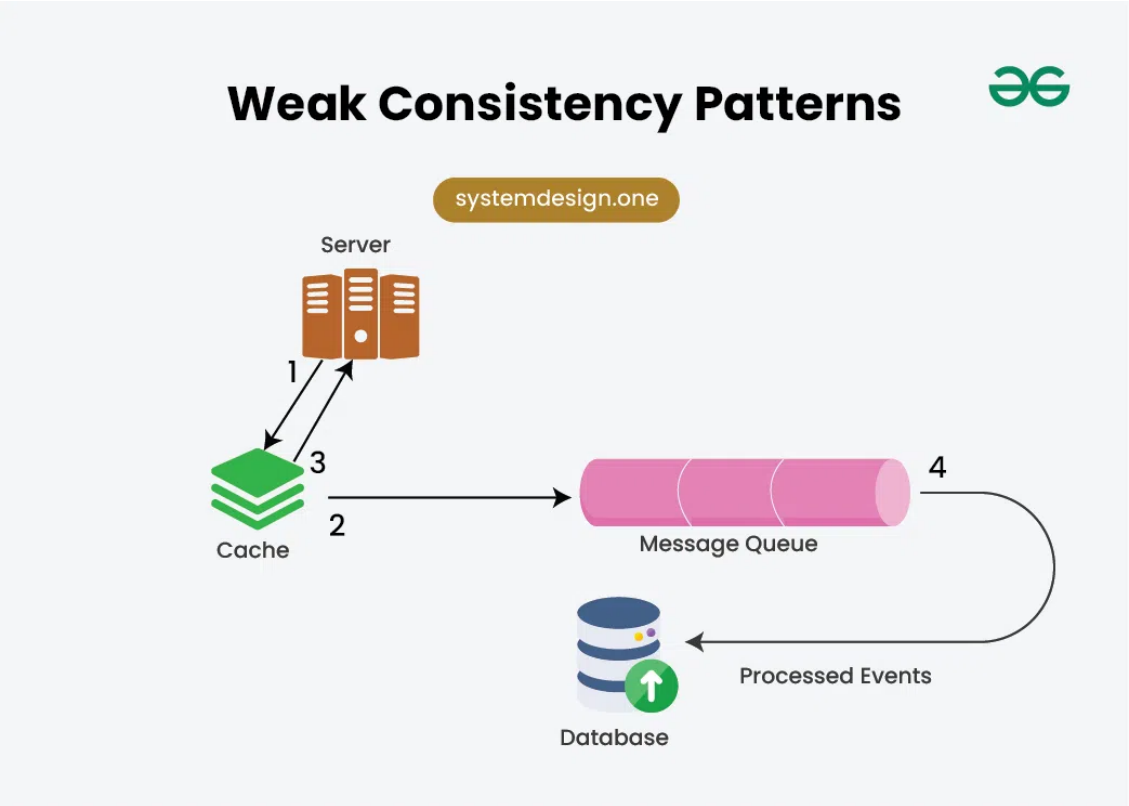
Операция, завершившаяся раньше по wall-clock, может оказаться позже в глобальном порядке.

Strong vs. Sequential Consistency

Свойство	**Strong**	**Sequential**
Единый порядок операций	Да	Да
Учет реального времени	Да	Нет
Влияние сетевых задержек	Выше	Ниже

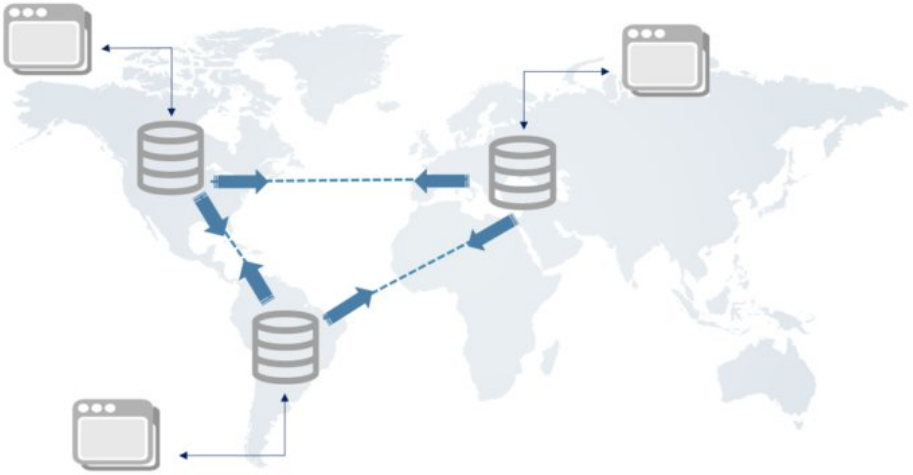
Грубо говоря: strong = sequential + real-time constraints.

Семейство моделей Weak Consistency



- Часто дают локальные гарантии:
  - Read Your Writes – видим результаты своих же записей
  - Monotonic Reads – новое чтение не вовращает старые значения
  - Monotonic Writes – новая запись побеждает старую запись
  - Writes Follow Reads – если прочитал значение, не перезапишешь его старым

Eventual Consistency



- Если новые записи прекратились, все реплики в итоге сходятся
- Не гарантирует срок сходимости
- На практике требуется:
  - анти-энтропия
  - детерминированный merge
  - контроль clock skew (для LWW)

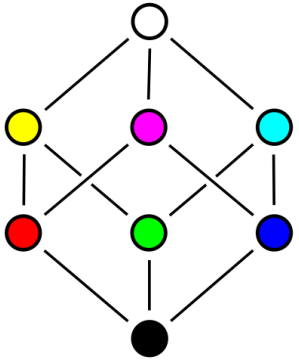
Типичный сценарий: кеши, каталоги, ленты рекомендаций.

Causal Consistency

- Сохраняет отношение happens-before между причинно связанными операциями
- Конкурентные (независимые) записи могут наблюдаться в разном порядке
- Пример:
  - пользователь A публикует пост
  - пользователь B отвечает на пост
  - нельзя увидеть ответ, не увидев исходный пост

Механизмы: vector clocks, dependency tracking.

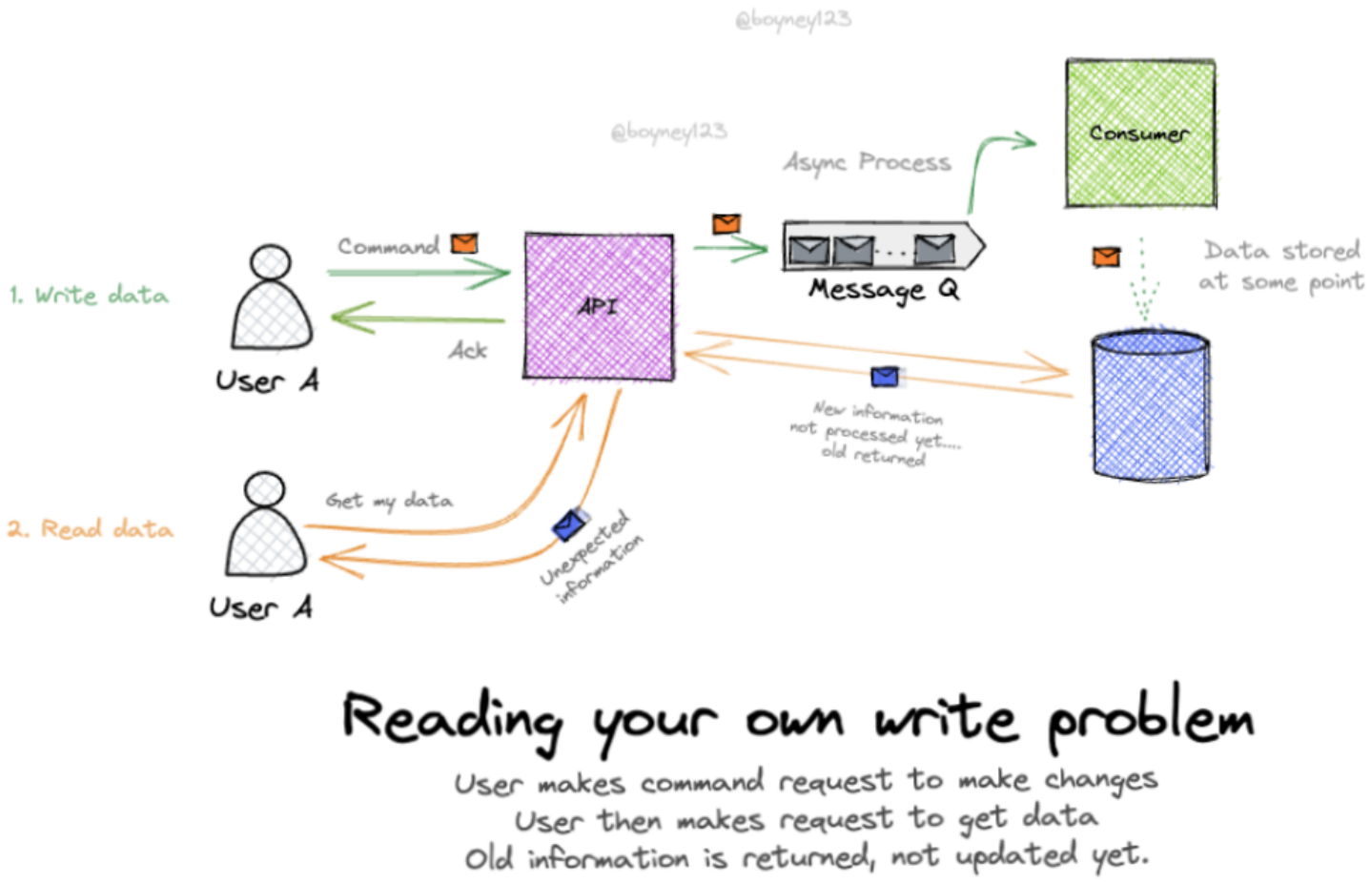
Strong Eventual Consistency



- При доставке одного и того же набора обновлений все реплики гарантированно равны
- Порядок доставки не важен
- Обычно достигается через CRDT:
  - операции коммутативны
  - merge ассоциативен, коммутативен и идемпотентен

Примеры: счетчики, множества, совместное редактирование документов.

Session Consistency



- Гарантии только внутри одной клиентской сессии (sticky sessions или session token с версией)

Как выбирать модель согласованности на практике

1. Выписать бизнес-инварианты (что **никогда** нельзя нарушить)
2. Разделить операции на критичные и некритичные
3. Назначить минимальную достаточную модель на операцию
   - платеж/списание: strong или serializable транзакции
   - лента/счетчики просмотров: eventual или causal
   - профиль пользователя: session guarantees часто достаточно
4. Контролировать production через p95/p99 latency и частоту конфликтов

Содержание

1 Модели согласованности

2 Сериализуемость транзакций

3 Транзакционность

Что такое серийное расписание?

Серийное расписание

Серийное расписание — это расписание, в котором транзакции выполняются по одной: сначала все операции одной транзакции, затем другой, без чередования.

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	
W(B)	
	R(B)
	W(B)
Commit	
	Commit

Non-serial

T1	T2
R(A)	
W(A)	
R(B)	
W(B)	
	R(A)
	W(A)
	R(B)
	W(B)
Commit	
	Commit

Serial

Что такое сериализуемое расписание?

Сериализуемое расписание

Расписание называется сериализуемым, если оно эквивалентно некоторому серийному расписанию.

T1	T2
R(A)	
	R(B)
W(A)	
	W(B)
Commit	
	Commit

Несерийное расписание

T1	T2
R(A)	
W(A)	
	R(B)
	W(B)
Commit	
	Commit

Серийное расписание

Оба расписания выше эквивалентны друг другу. Они сериализуемые.

Конфликтно-сериализуемость

Определение

Расписание называется конфликтно-сериализуемым, если его можно превратить в серийное, переставляя только неконфликтующие операции.

Две операции конфликтуют, если:

- они из разных транзакций
- работают с одним и тем же объектом
- хотя бы одна из них — запись (write)

Конфликт:

- T_1: write(A)
- T_2: read(A)

Нет конфликта:

- T_1: read(A)
- T_2: read(A)

Конфликтно-сериализуемость

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	
W(B)	
	R(B)
	W(B)
Commit	
	Commit

Before swapping

T1	T2
R(A)	
W(A)	
R(B)	
W(B)	
	R(A)
	W(A)
	R(B)
	W(B)
Commit	
	Commit

After swapping

***Non-conflicting operations**

***Conflict Serializability**

Разбор задачи: формулировка

Даны две транзакции:

```
T1:
  a = A.getBalance();
  A.credit(INTEREST * a);

T2:
  A.debit(100);
  B.credit(100);
```

- Предположение: каждая операция на объекте атомарна
- Нужно: перечислить все interleavings и для каждого ответить:
  - это serial execution?
  - это conflict-serializable execution?

Interleaving #1

Шаг	T_1	T_2
1	`A.getBalance()`	
2	`A.credit(INTEREST * a)`	
3		`A.debit(100)`
4		`B.credit(100)`

Interleaving #1

Шаг	T_1	T_2
1	`A.getBalance()`	
2	`A.credit(INTEREST * a)`	
3		`A.debit(100)`
4		`B.credit(100)`

Serial: да ($T_1 \to T_2$).

Interleaving #1

Шаг	T_1	T_2
1	`A.getBalance()`	
2	`A.credit(INTEREST * a)`	
3		`A.debit(100)`
4		`B.credit(100)`

Serial: да ($T_1 \to T_2$). **Conflict-serializable:** да.

Interleaving #2

Шаг	T_1	T_2
1	`A.getBalance()`	
2		`A.debit(100)`
3	`A.credit(INTEREST * a)`	
4		`B.credit(100)`

Interleaving #2

Шаг	T_1	T_2
1	`A.getBalance()`	
2		`A.debit(100)`
3	`A.credit(INTEREST * a)`	
4		`B.credit(100)`

Serial: нет.

Interleaving #2

Шаг	T_1	T_2
1	`A.getBalance()`	
2		`A.debit(100)`
3	`A.credit(INTEREST * a)`	
4		`B.credit(100)`

Serial: нет. **Conflict-serializable:** нет (цикл $T_1 \leftrightarrow T_2$).

Interleaving #3

Шаг	T_1	T_2
1	`A.getBalance()`	
2		`A.debit(100)`
3		`B.credit(100)`
4	`A.credit(INTEREST * a)`	

Interleaving #3

Шаг	T_1	T_2
1	`A.getBalance()`	
2		`A.debit(100)`
3		`B.credit(100)`
4	`A.credit(INTEREST * a)`	

Serial: нет.

Interleaving #3

Шаг	T_1	T_2
1	`A.getBalance()`	
2		`A.debit(100)`
3		`B.credit(100)`
4	`A.credit(INTEREST * a)`	

Serial: нет. **Conflict-serializable:** нет (цикл $T_1 \leftrightarrow T_2$).

Interleaving #4

Шаг	T_1	T_2
1		`A.debit(100)`
2	`A.getBalance()`	
3	`A.credit(INTEREST * a)`	
4		`B.credit(100)`

Interleaving #4

Шаг	T_1	T_2
1		`A.debit(100)`
2	`A.getBalance()`	
3	`A.credit(INTEREST * a)`	
4		`B.credit(100)`

Serial: нет.

Interleaving #4

Шаг	T_1	T_2
1		`A.debit(100)`
2	`A.getBalance()`	
3	`A.credit(INTEREST * a)`	
4		`B.credit(100)`

Serial: нет. **Conflict-serializable:** да (эквивалент $T_2 \to T_1$).

Interleaving #5

Шаг	T_1	T_2
1		`A.debit(100)`
2	`A.getBalance()`	
3		`B.credit(100)`
4	`A.credit(INTEREST * a)`	

Interleaving #5

Шаг	T_1	T_2
1		`A.debit(100)`
2	`A.getBalance()`	
3		`B.credit(100)`
4	`A.credit(INTEREST * a)`	

Serial: нет.

Interleaving #5

Шаг	T_1	T_2
1		`A.debit(100)`
2	`A.getBalance()`	
3		`B.credit(100)`
4	`A.credit(INTEREST * a)`	

Serial: нет. **Conflict-serializable:** да (эквивалент $T_2 \to T_1$).

Interleaving #6

Шаг	T_1	T_2
1		`A.debit(100)`
2		`B.credit(100)`
3	`A.getBalance()`	
4	`A.credit(INTEREST * a)`	

Interleaving #6

Шаг	T_1	T_2
1		`A.debit(100)`
2		`B.credit(100)`
3	`A.getBalance()`	
4	`A.credit(INTEREST * a)`	

Serial: да ($T_2 \to T_1$).

Interleaving #6

Шаг	T_1	T_2
1		`A.debit(100)`
2		`B.credit(100)`
3	`A.getBalance()`	
4	`A.credit(INTEREST * a)`	

Serial: да ($T_2 \to T_1$). **Conflict-serializable:** да.

Итоги по задаче

#	Serial	Conflict-serializable
1	да	да
2	нет	нет
3	нет	нет
4	нет	да
5	нет	да
6	да	да

- Всего interleavings: 6
- Serial executions: **#1, #6** (2 из 6)
- Conflict-serializable: **#1, #4, #5, #6** (4 из 6)
- Не conflict-serializable: **#2, #3**

Как проверять быстро

Для каждого расписания достаточно выписать конфликтные пары на A и проверить, возник ли цикл между T_1 и T_2.

Содержание

1. Модели согласованности
2. Сериализуемость транзакций
3. Транзакционность

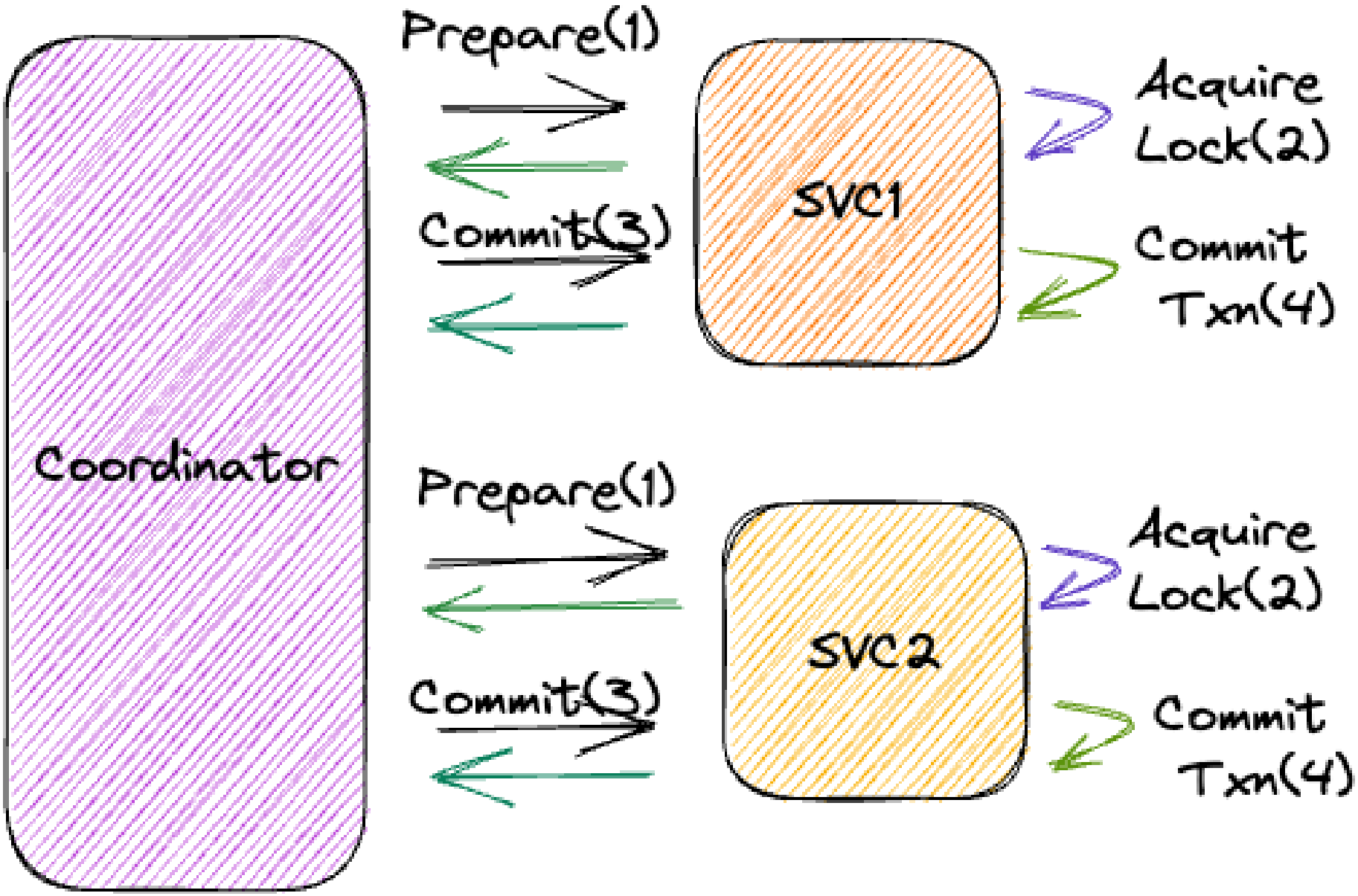
Транзакционная модель ACID

- **Atomicity**: транзакция либо целиком применена, либо откат
- **Consistency**: переход между корректными состояниями БД
- **Isolation**: параллельные транзакции не мешают и не заметны друг другу
- **Durability**: после commit данные переживают сбой узла (WAL)

Важно

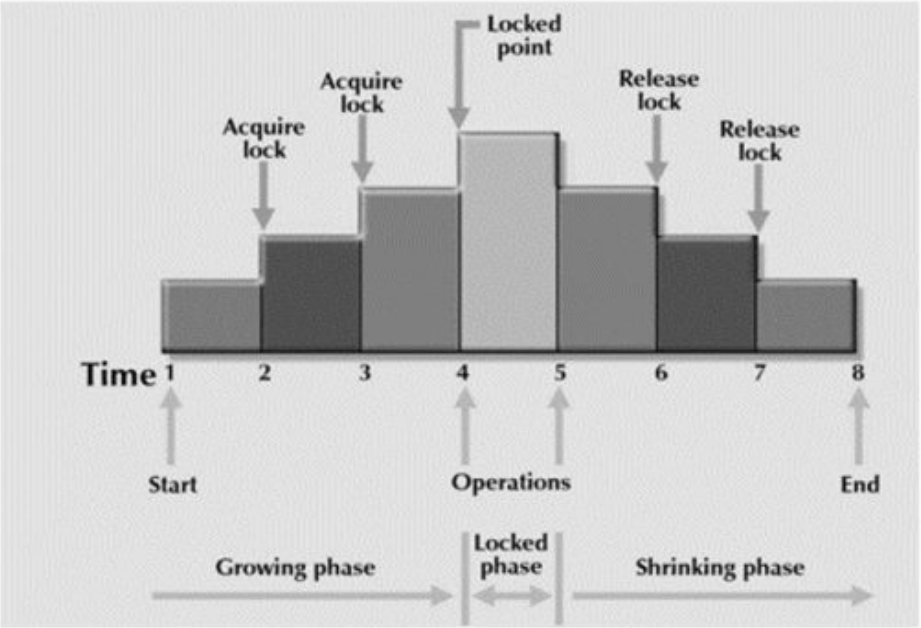
ACID — это свойства *модели выполнения транзакций*, а не «strong consistency». Но сам подход является пессимистическим.

2PC: 2-Phase Commit



2PL: 2-Phase Locking

- This protocol ensures conflict-serializable schedules.
- Phase 1: Growing Phase
  - Transaction may obtain locks
  - Transaction may not release locks
- Phase 2: Shrinking Phase
  - Transaction may release locks
  - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).



Задача 1: ACID и 2PL

Если мы просто связываем блокировки (locks) с объектами в транзакционной системе, чтобы реализовать двухфазную блокировку (2PL),

- какие свойства ACID выполняются автоматически,
- а какие требуют дополнительной реализации?

Почему?

Разбор задачи 1

Разбор задачи 1

Atomicity (Атомарность) — требует доп. механизмов

- при сбое изменения могут остаться частично применёнными
- нужны undo-логи / rollback

Разбор задачи 1

Atomicity (Атомарность) — требует доп. механизмов

- при сбое изменения могут остаться частично применёнными
- нужны undo-логи / rollback

Consistency (Согласованность) — не гарантируется самим 2PL

- если транзакция сама нарушает инварианты, 2PL это не предотвратит
- согласованность обеспечивается корректностью кода и ограничениями БД

Разбор задачи 1

Atomicity (Атомарность) — требует доп. механизмов

- при сбое изменения могут остаться частично применёнными
- нужны undo-логи / rollback

Consistency (Согласованность) — не гарантируется самим 2PL

- если транзакция сама нарушает инварианты, 2PL это не предотвратит
- согласованность обеспечивается корректностью кода и ограничениями БД

Isolation (Изолированность) — обеспечивается

- поскольку гарантирует конфликтную сериализуемость

Разбор задачи 1

Atomicity (Атомарность) — требует доп. механизмов

- при сбое изменения могут остаться частично применёнными
- нужны undo-логи / rollback

Consistency (Согласованность) — не гарантируется самим 2PL

- если транзакция сама нарушает инварианты, 2PL это не предотвратит
- согласованность обеспечивается корректностью кода и ограничениями БД

Isolation (Изолированность) — обеспечивается

- поскольку гарантирует конфликтную сериализуемость

Durability (Долговечность) — требует доп. механизмов

- 2PL не гарантирует сохранение данных после commit
- данные могут быть потеряны при сбое (например, в памяти)
- нужны WAL и запись на диск

Задача 2: Durability и Atomicity

Свойство **Durability (долговечность)** гарантирует, что если транзакция объявлена зафиксированной (committed), то её результаты сохранятся даже при сбоях (например, отключении питания).

Однако очевидное решение — просто записывать изменения объектов на диск перед подтверждением транзакции — может привести к проблемам с **атомарностью**.

Вопрос: При каких обстоятельствах это может произойти?

Ответ: Durability vs Atomicity

Проблема: простая запись данных на диск перед commit

Когда возникает нарушение атомарности:

- транзакция изменяет несколько объектов
- изменения пишутся на диск **по частям**
- происходит сбой (crash, power loss) **в процессе записи**

Что получаем:

- часть изменений уже записана
- часть — нет

Итог:

- транзакция не завершена
- но её эффекты частично применены
- нарушается **Atomicity (не "всё или ничего")**

Корень проблемы: нет механизма отката или восстановления

Write-Ahead Logging (WAL)

Идея: сначала пишем в лог, потом в данные

Правила WAL:

- **1. Log before data**
  - изменения сначала записываются в лог
  - только потом — в базу
- **2. Commit after log flush**
  - commit разрешён только после записи лога на диск

Что хранится в логе:

- старое значение (undo)
- новое значение (redo)

При сбое:

- незавершённые транзакции $\rightarrow$ **UNDO**
- завершённые, но не записанные $\rightarrow$ **REDO**

Задача 3: Двухфазная блокировка (2PL)

2PL

Двухфазная блокировка (2PL) — это один из способов обеспечения изоляции при параллельном выполнении транзакций над несколькими объектами.

Она состоит из двух фаз:

- фаза роста (lock expansion)
- фаза сжатия (lock contraction)

Планировщик

T_1:

- $x = A.read()$
- $B.write(x)$

T_2:

- $x = B.read()$
- $C.write(x)$

T_3:

- $x = C.read()$
- $A.write(x)$

Предположим, что планировщик транзакций, стремясь обеспечить справедливость, использует round-robin планирование по строкам кода.

Время	T_1	T_2	T_3
1	$x = A.read()$		
2		$x = B.read()$	
3			$x = C.read()$
4	$B.write(x)$		
5		$C.write(x)$	
6			$A.write(x)$

Если транзакция заблокирована (ждёт lock), планировщик пропускает её и переходит к следующей.

Задача 3а

Если использовать **наивную 2PL** для транзакций T_1, T_2, T_3:

Вопрос: Какое расписание будет получено?

Решение задачи 3а

Время	T_1	T_2	T_3
1	lock(A), read(A)		
2		lock(B), read(B)	
3			lock(C), read(C)
4	lock(B), blocked		
5		lock(C), blocked	
6			lock(A), blocked

Результат: циклическая ждёт-граф: $T_1 \to T_2 \to T_3 \to T_1$.

Вывод: наивная 2PL приводит к **deadlock**.

Задача 3b

Взаимная блокировка (deadlock) — серьёзная проблема для 2PL.

Предположим, что система реализует простой детектор дедлоков: если есть активные транзакции, но ни одна не может быть выполнена, тогда все активные транзакции прерываются (abort), их блокировки освобождаются, и они перезапускаются в следующей итерации

Вопрос: Какое расписание возникнет в этом случае?

Решение задачи 3b

Время	T_1	T_2	T_3
1	lock(A), read(A)		
2		lock(B), read(B)	
3			lock(C), read(C)
4	lock(B), blocked		
5		lock(C), blocked	
6			lock(A), blocked
7	abort, все откатываются		
8	lock(A), read(A)		
9		lock(B), read(B)	
10			lock(C), read(C)
11	lock(B), blocked		
12		lock(C), blocked	
13			lock(A), blocked

Проблема: система входит в **бесконечный цикл**.

Вывод: простое обнаружение дедлока + перезапуск приводит к **livelock**.

Задача 3c

Livelock также может возникать при наивном разрешении дедлоков.

Вопрос: Как можно изменить схему из пункта (b), чтобы гарантировать прогресс, если дедлок обнаружен?

Решение задачи 3c: Приоритеты

Идея: использовать **приоритеты** или **временные метки** для упорядочения транзакций.

Это предотвращает livelock, обеспечивая **детерминированный** порядок разрешения конфликтов.

Рассмотрим две основные схемы:

1. **Wait-Die**: старая транзакция ждёт молодую, молодая откатывается
2. **Wound-Wait**: старая откатывает молодую, молодая ждёт старую

Решение 3c: Wait-Die схема

Правило: если T_i (старше) конфликтует на lock с T_j (моложе):

- T_i ждёт
- T_j откатывается (dies)

Время	T_1	T_2	T_3
1	lock(A), read(A)		
2		lock(B), read(B)	
3			lock(C), read(C)
4	lock(B), ждёт		
5		lock(C), ждёт	
6			lock(A), dies!
7			(перезапуск)
8		write(C) ✓	
9		(завершено)	
10	write(B) ✓		
11	(завершено)		

Результат: гарантирован прогресс! Нет livelock.

Решение 3c: Wound-Wait схема

Правило: если T_i (старше) конфликтует на lock с T_j (моложе):

- T_i прерывает (wounds) T_j
- T_j откатывается
- T_i получает lock

Время	T_1	T_2	T_3
1	lock(A), read(A)		
2		lock(B), read(B)	
3			lock(C), read(C)
4	lock(B), wounds T_2		
5		(dies)	
6		(перезапуск)	
7			lock(A), wounds T_3
8			(dies)
9			(перезапуск)
10	write(B) ✓		
11	(завершено)		

Результат: также гарантирован прогресс, старые транзакции

Задача 3d

Предположим, что мы знаем, что будем постоянно исполнять транзакции T_1, T_2, T_3 и хотим объединить их в одну, чтобы исключить livelock.

Вопрос: Как мы можем это сделать, и почему это будет гарантировать отсутствие дедлоков?

Итоги

- (!) Через неделю дедлайн по второму дз
- Обсудили модели согласованности, сериализуемость и транзакционность